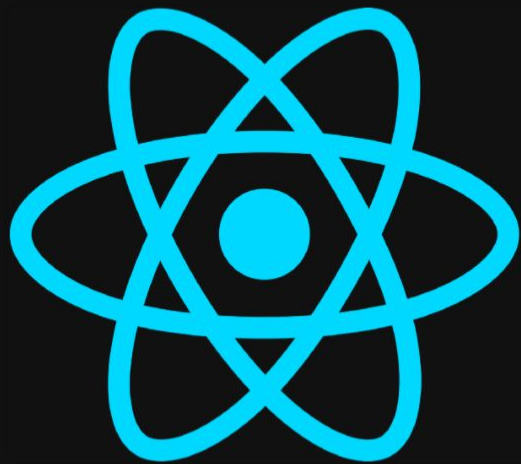




React



React Core Principles

User Interfaces



Hard to build because there is so much **state**

Data changing over time is the **root of all evil**

What is React?

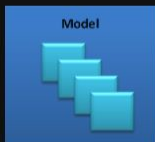


- A Library for creating User Interfaces
- Uses a Virtual DOM that selectively renders subtrees of nodes based upon state changes
- Renders UI and responds to events
- People say it is the V in MVC...
 - I agree the **Model is missing**
 - And it is **mostly about the View**
 - But my **Controller is somewhere** in there!

Separation of Concerns



Design principle for separating a computer program into distinct sections, such that each section addresses a separate concern



Models are usually plain JavaScript objects

Views are usually implemented using some sort of HTML template system



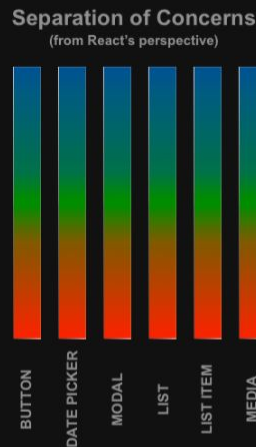
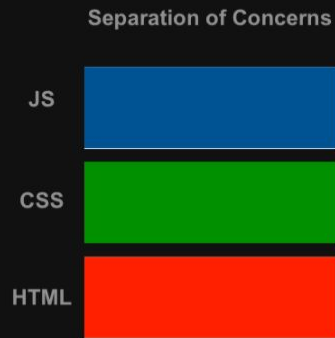
Controllers contain view related logic tightly coupled with Views

Most JavaScript MVC frameworks end up separating by technology and not by concerns!

View and Controllers are tightly coupled, making it difficult to change one without the changing the other

Separation of Components

React uses Components to separate concerns, written with the full power of JavaScript.

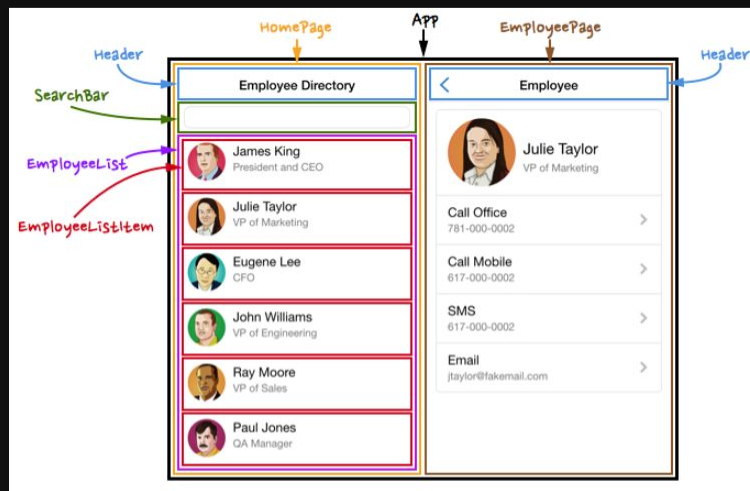


In React everything is a Component!

Component Based



React is all about building components, not templates
They split the UI into independent reusable pieces



Components are reusable, composable, maintainable, testable and know how to render themselves into the DOM

Composition



Just like regular functions, React components can be composed together

```
<Parent />
```

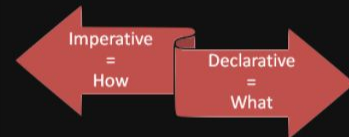
```
<Child />
```

```
<Child />
```

```
<Child />
```

This principle is a key feature of React and present at the core of how react applications are developed

Declarative User Interface



- **Imperative Programming** - sequence of instructions telling the machine exactly how to do something
- **Declarative Programming** - telling the machine what should happen and let it work out how to do it

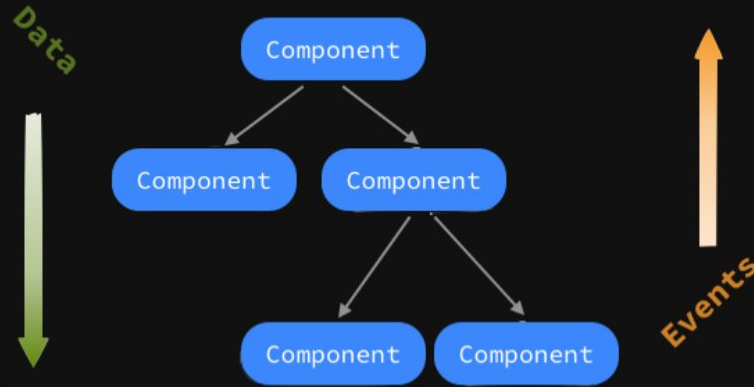
```
// Imperative jQuery code
// to create a header section
function createHeaderSection(text) {
  let element = $('<div/>');
  let header = $('<h1/>');
  header.text(text);
  element.append(header);
  return element;
}
```

```
// Declarative React code to
// create a header section component
function HeaderBlock(props) {
  return (
    <div>
      <h1>{props.text}</h1>
    </div>
  );
}
```

React's declarative style leads to more readable code and with less bugs

Unidirectional Data Flow

React implements one-way reactive data flow



- Components are rendered as a function of the application state
- User events can trigger explicit state mutations
- State mutations will re-render components

One way data flow is deterministic, making it easier to predict and reason about state changes and data flows

Virtual DOM



Re-render on every state change seems expensive, but is **fast** because it happens on a **Virtual DOM**.

On every update the following process takes place:

1. New virtual DOM subtree is built
2. Diff between new and old is performed
3. Minimal set of DOM mutations is calculated
4. DOM mutations are queued
5. All updates are executed together in a single batch

The DOM is slow, but React is fast!

The React Way

- **Components**, not templates
- **Re-render**, don't mutate
- Virtual DOM is **simple** and **fast**

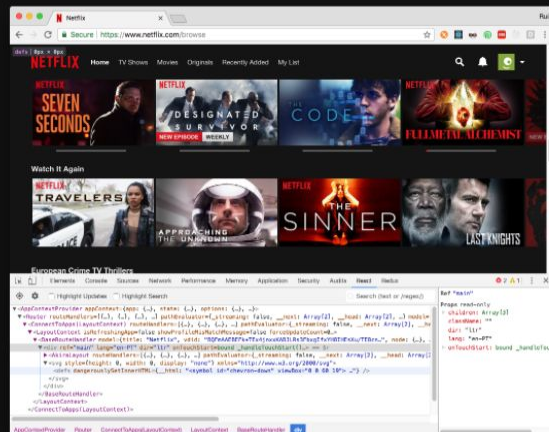




React Ecosystem

React Developer Tools

An extension that allows inspection of React component hierarchy in the Chrome and Firefox Developer Tools



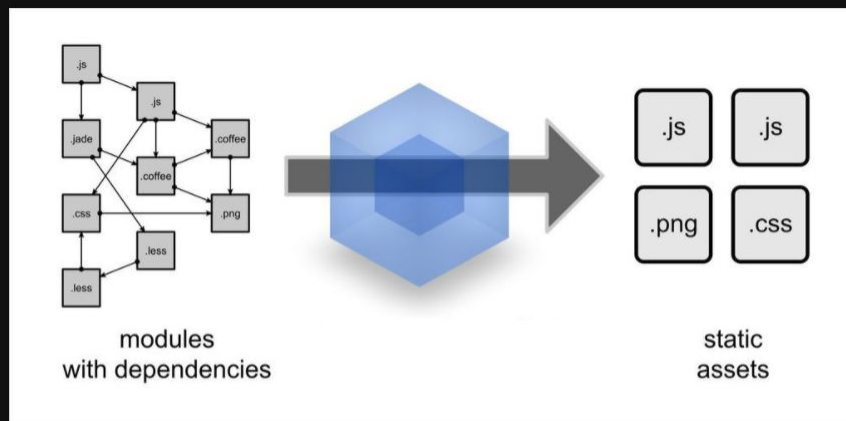
Its easy to inspect component hierarchy of existing React applications

Webpack



Static module bundler for modern JavaScript applications

Recursively builds a dependency graph that includes every module in the application and packages them into one or more bundles



Webpack Configuration



Webpack can be configured around four core concepts:

- **Entry** point indicates which module webpack use to begin building its dependency graph
- **Output** property tells webpack where to emit the bundles
- **Loaders** enable webpack to convert to modules and process all types of files
- **Plugins** can be used to perform a wide variety of tasks such as optimization and minification

```
// webpack config uses common.js node modules, not ES6
module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist') // absolute path required
  },

  // module transformations to apply on files
  module: {
    // loaders to apply if files match the test regex
    rules: [
      // css-loader: transform all css files in valid modules
      // style-loader: add css to dom
      { test: /\.css$/, use: ['style-loader', 'css-loader'] }
    ]
  },

  // plugins to be invoked by webpack
  plugins: [new HtmlWebpackPlugin({ template: 'public/index.html' })]
};
```

{ Hands On }

Webpack Build System

BABEL

Requires **plugins** to perform transformations on code

```
{
  // webpack config file
  module: {
    rules: [
      // Babel webpack plugin
      { test: /\.js$/, exclude: /node_modules/, loader: "babel-loader" }
    ]
  }
}
```

Babel **presets** apply a predefined set of plugins

```
# react dependencies
$ npm install --save react react-dom

# webpack babel dependencies
$ npm install --save-dev babel-core babel-loader babel-preset-env babel-preset-react

{
  // babel config in package.json file
  "babel": { "presets": ["env", "react"] }
}
```

Create React App

Official CLI for creating react applications with no setup



Welcome to React

To get started, edit `src/App.js` and save to reload.

```
# Execute directly from the NPM registry
$ npx create-react-app my-app

# Run the application in development mode
$ cd my-app
$ npm start
```

Axios

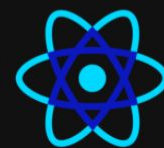


Promise based HTTP client for the browser and Node.js

- Automatic JSON transforms
- XSRF protection
- Request canceling

```
async function getUser() {  
  try {  
    const response = await axios.get('/user?ID=12345');  
    console.log(response);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Quite popular on the React community and a better alternative to fetch



React Basics

React Elements



Immutable object describing what should be rendered to the screen and containing **two** main properties:

- **type** - String for DOM node or ReactClass for a react component
- **props** - element attributes

```
{
  type: 'div',
  props: {
    className: 'container',
    children: '<Academia de Código_>'
  }
}
```

```
{
  type: Button,
  props: {
    className: 'ui primary button',
    children: 'Login'
  }
}
```

```
<!-- DOM Node -->
<div class="container">
  &lt;Academia de Código_&gt;
</div>
```

```
<!-- React Component -->
<Button className="ui primary button">
  Login
</Button>
```

Elements can be created calling **React.createElement(node, props)**

Child props can be passed as arguments **React.createElement(node, props, child1, child2, ...)**

React Components

A React Component is a function or a class which accepts input properties and returns a React Element

```
// Button Component
function Button(props) {
  return React.createElement('button', props);
}
```

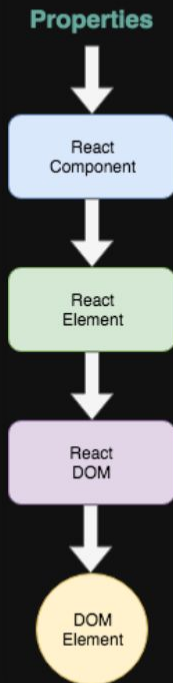
React Components can be rendered by calling **ReactDOM.render(Component(), node)**

```
// properties to pass to the Button component
const props = { className: 'ui button primary', children: 'Login' };

// create element from Button component
const element = React.createElement(Button, props);

// render element to the root div
ReactDOM.render(element, document.getElementById('root'));
```

React Components must act like pure functions with respect to their props and do not attempt to change them



{ Hands On }

Rendering React Components

JSX



Syntax extension to JavaScript to describe React Elements

Syntactic sugar for **React.createElement(component, props, ...children)**

```
// required for jsx
import React from 'react';

const type = 'section';
const element = (
  <div className="{type}">
    <h1>I am a component!</h1>
    <h2>I am a sub title!</h2>
  </div>
);
```

```
// required for createElement
import React from 'react';

const element = React.createElement('div',
  { className: 'section',
    React.createElement('h1', null, 'I am a component!'),
    React.createElement('h1', null, 'I am a sub title!')
  }
);
```

Instead of artificially separating technologies by putting markup and logic in separate files,
React separates concerns with loosely coupled components that contain both

Inline Styles

The **style** attribute can be used to style a React component

An object with camelCased properties is used, not strings

```
// paragraph style object
const pStyle = {
  color: 'red'
};

// styled react element
const alertText = <p style={pStyle} />;

// render red paragraph with no CSS
ReactDOM.render(alertText, document.getElementById('root'));
```

Inline styles are normally used for dynamic computed styles as CSS classes are generally more efficient

{ Hands On }

Rendering JSX Components



React Components

Functional Components



Contain no state and expressed as a single pure function which accepts a **props** object and returns a React **element**

```
// Declare the React Welcome Component
function Welcome(props) {
  return <h2>Welcome {props.name}</h2>;
}

// Instantiate and render a React Element
ReactDOM.render(<Welcome name="Rui" />, document.getElementById('root'));

// Same as above
const element = Welcome({name: 'Rui'});
ReactDOM.render(element, document.getElementById('root'));
```

Functional components are stateless and pure, for the same props they always return the same element

React components should start with uppercase (**<Welcome />**) to distinguish them from regular DOM tags(**<div />**)

Composing Components



Complex UI does not fit in a single function and needs to be abstracted into reusable pieces, hiding implementation details

JSX makes it really easy to compose components together

```
function ListItem(props) {  
  return <li>{props.item}</li>;  
}  
  
function Popular() {  
  return (  
    <ul>  
      <ListItem item="JavaScript" />  
      <ListItem item="Java" />  
      <ListItem item="Ruby" />  
    </ul>  
  );  
}  
  
ReactDOM.render(<Popular />, document.getElementById('root'));
```

When a part of the UI is used several times or is complex enough on its own, **it should be extracted into a reusable component.**

Compositional Components



Wrapper Components can be created using the **children** property

```
// compose by passing props down
function PrimaryButton(props) {
  return (
    <button>
      {props.icon}{props.text}
    </button>
  )
}

// usage:
<PrimaryButton
  icon={IconFile} text="Login"
/>
```

```
// compose using children
function PrimaryButton(props) {
  return (
    <button>
      {props.children}
    </button>
  )
}

// usage:
<PrimaryButton>
  <IconFile/> Login
</PrimaryButton>
```

This technique provides a more robust and flexible way of composing than passing props down the component hierarchy

Specially useful when components do not know their children ahead of time, such as generic Modal, Sidebar or Dialog boxes for example.

Property Typechecking



React includes a property typechecking library to help catching bugs during development

```
import PropTypes from 'prop-types';

function ListItem({ item , color }) {
  return <li style={color: color}>{item}</li>;
}

ListItem.propTypes = {
  item: PropTypes.string.isRequired,
  color: PropTypes.string
};

// optional properties should have defaults
ListItem.defaultProps = {
  color: 'red'
};
```

prop types and **default values** checks are enforced during development but removed from production code

Lists in React



We can create lists in React using JSX and plain JavaScript array methods such as **map** or **filter**

```
function ListItem({ item }) {  
  return <li>{item}</li>;  
}  
  
function Popular() {  
  const languages = ['All', 'JavaScript', 'Java', 'HTML', 'CSS', 'C', 'C++'];  
  return (  
    <ul>  
      { languages.map(lang => <ListItem key={lang} item={lang} />) }  
    </ul>  
  );  
}  
  
ReactDOM.render(<Popular />, document.getElementById('root'))}
```

Array items require special unique **key** attribute for react to identify items changed, added or removed

Contrary to other JavaScript frameworks, there is no special syntax, its all JavaScript with JSX sugar on top

Class Components



A Class component can be created by extending from
React.Component

```
class Popular extends React.Component {  
  render() {  
    return (  
      <ul>  
        { languages.map(lang => <ListItem key={lang} item={lang} />) }  
      </ul>  
    );  
  }  
}  
  
ReactDOM.render(<Popular />, document.getElementById('root'));
```

Class components without state can be transformed in functional components
by moving the contents of the **render()** method into a separate function

The Render method needs to act as a pure function, it should **only read state and props and render a UI**

Component State



State allows React components to change their output over time

```
function ListItem({ item, color }) {  
  return <li style={{ color }}>{item}</li>;  
}  
  
class Popular extends React.Component {  
  static languages = ['All', 'JavaScript', 'Java', 'HTML', 'CSS', 'C', 'C++'];  
  state = { selected: 'All' };  
  
  selectLanguage(lang) {  
    this.setState({ selected: lang });  
  }  
  
  render() {  
    return (  
      <ul>  
        {Popular.languages.map(lang => {  
          let color = lang === this.state.selected ? 'red' : 'black';  
          return <ListItem color={color} key={lang} item={lang} />;  
        })}  
      </ul>  
    );  
  }  
}
```

Component state lives in a special **state** property, explicitly updated by invoking the **setState()** method

Updating the state causes the component to re-render

Updating State



Updates to the UI are achieved through state updates, which are enqueued, batched and delayed by React

```
class Counter extends React.Component {
  state = { value: 0 };

  stateUpdateCb() { console.log('New state is ', this.state); }

  // new state does not depend on previous state, shallow merge is performed
  reset() {
    this.setState({ value: 0 }, this.stateUpdateCb);
  }

  // new state depends on previous state, state updater function is recommended
  increment() {
    this.setState(
      prevState => ({ value: prevState.value + 1 }),
      this.stateUpdateCb
    );
  }

  render() {
    return <h2>{this.state.value}</h2>;
  }
}
```

State updates have to be explicitly triggered through `setState()`, it is not possible to update `this.state` directly

Event Handling



Event handlers receive instances of **SyntheticEvent**, a cross-browser wrapper around the native events

```
class Counter extends React.Component {
  state = { value: 0 };

  stateUpdateCb() { console.log('New state is ', this.state); }

  increment = () => {
    this.setState(
      prevState => ({ value: prevState.value + 1 }),
      this.stateUpdateCb
    );
  };

  render() {
    return <h2 onClick={this.increment}>{this.state.value}</h2>;
  }
}
```

React events are named using camelCase and not lowercase like DOM events

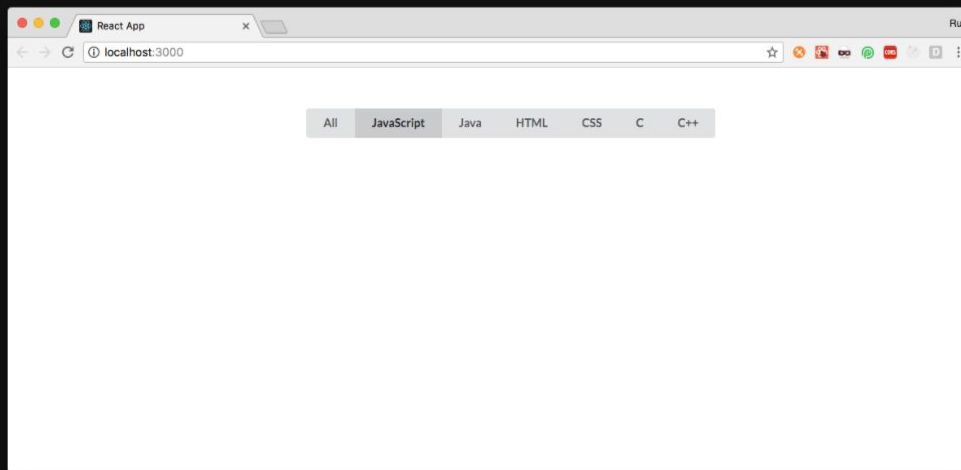
SyntheticEvent is reused and all properties nullified for performance, making it unusable in asynchronous callbacks

{ Exercise }

Clickable React Counter

{ Exercise }

Popular Programming Languages Clickable Menu



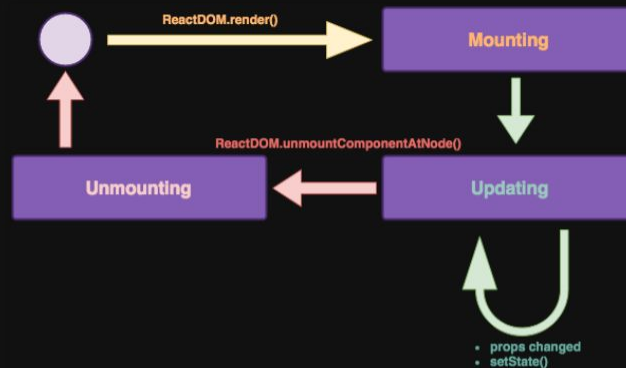
Component Life Cycle



React components provide special methods for hooking into the component life cycle

Most life cycle methods follow into one of these categories:

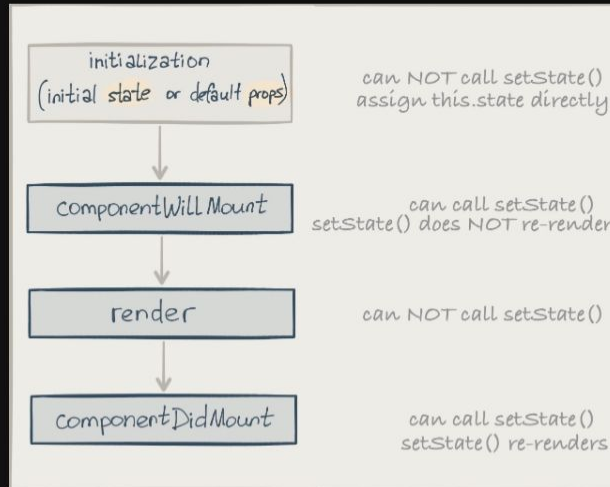
1. Component gets mounted into the DOM
2. Component receives new data
3. Component gets unmounted from the DOM



Mounting Phase



Component is instantiated and inserted into the DOM

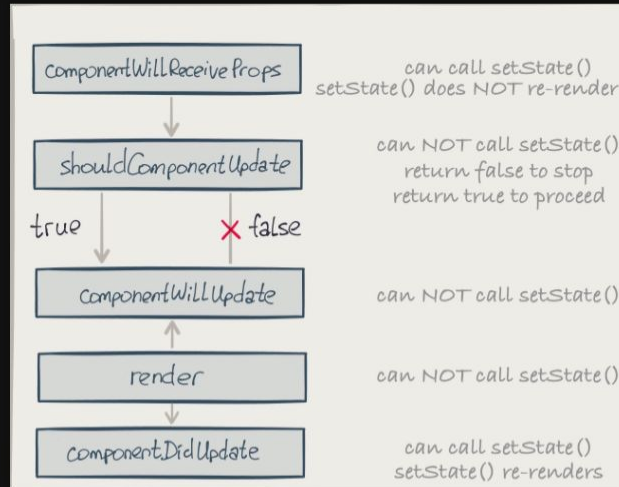


componentDidMount() is the correct place to put any initialization code such as starting timers or fetching data using AJAX

Updating



Component is being re-rendered due to changes in props or state



shouldComponentUpdate() can prevent unnecessary re-rendering and boost performance

Component Re-render



When the rendering process finds a difference in a component it re-renders the whole subtree

```
function NothingChanges() {  
  console.log('Will render one more time...');  
  return <h3>Nothing changed here...</h3>;  
}  
  
class Counter extends React.Component {  
  state = { value: 0 };  
  
  increment = () => {  
    this.setState(  
      prevState => ({ value: prevState.value + 1 })  
    );  
  };  
  
  render() {  
    return (  
      <div>  
        <h2 onClick={this.increment}>{this.state.value}</h2>  
        <NothingChanges />  
      </div>  
    );  
  }  
}
```

NothingChanges component will be render every time its parent component renders

How do we avoid these unnecessary re-renders?

Unmounting



Component is being removed from the DOM

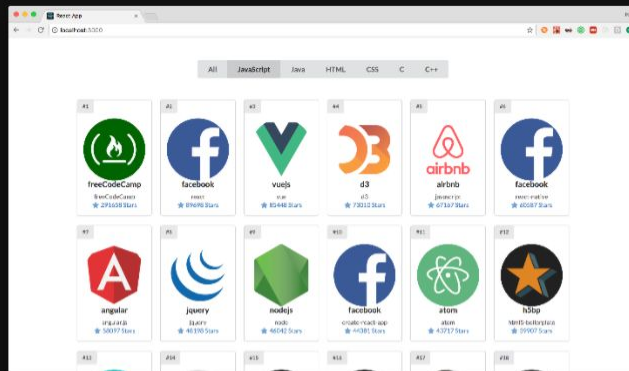
componentWillUnmount

can NOT call setState()

componentWillUnmount() is where cleanup code such as invalidating timers and canceling ajax requests should go

{ Exercise }

Popular Programming Languages on GitHub



Forms in React



HTML form elements such as **input**, **textarea** or **select**, maintain their state - user input - in the DOM

React provides two different approaches to form handling:

- **Uncontrolled Components** - they work just like traditional HTML forms, keeping and updating state in the DOM
- **Controlled Components** - form state is managed by React just like every other piece of state in the application

Uncontrolled Components



The browser stores and updates the value for each form input element

Whenever the value is needed it has to be pulled from the field

```
class Form extends React.Component {
  handleSubmit = () => {
    const value = this.input.value;
    // do something with value
  };

  render() {
    return (
      <div>
        <input type="text" ref={input => (this.input = input)} />
        <button onClick={this.handleSubmit}>Sign Up</button>
      </div>
    );
  }
}
```

The **ref** attribute receives a callback which is called with a reference to the DOM node when the component is mounted

Uncontrolled inputs should be used when converting a preexisting codebase to React or when integrating with a non-React library

Controlled Components



The **React way** for doing form handling,
where the React state is the single source of truth

Input values are pushed into form fields using a value from state or props

```
class Form extends React.Component {
  state = { text: '' };

  handleChange = event => {
    this.setState({ text: event.target.value });
  };

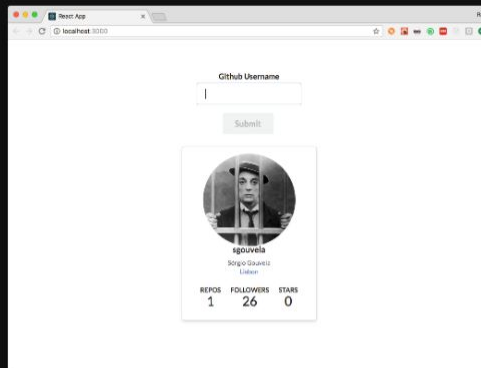
  handleSubmit = () => {
    // do something with this.state.text
  };

  render() {
    return (
      <div>
        <input type="text" value={this.state.text} onChange={this.handleChange} />
        <button onClick={this.handleSubmit}>Sign up</button>
      </div>
    );
  }
}
```

Every state mutation will have an associated handler function, making it really easy to modify or validate user input

{ Exercise }

Developer Profile on GitHub



React Router



Provides a collection of navigational components,
composed declaratively in React applications

Everything is a component in React Router,
which means routing occurs when the app is rendering

In static routing, routes are declared before rendering takes place,
which is exactly the opposite of React Router **dynamic routing**

Rendering a Router



The Router receives a single child component which should render the rest of the application

```
// use the browser based version
import { BrowserRouter } from 'react-router-dom';

ReactDOM.render((
  <BrowserRouter>
    <App />
  </BrowserRouter>
), document.getElementById('root'));
```

A **history** object is available to child components and used to keep track of the current location and re-render the application when it changes

Route Matching



A **Route** component will render its content when its **path** prop matches the current location.pathname

```
import { Route } from 'react-router-dom';

// when location = { pathname: '/about' }
<Route path="/" component={Header}/> // renders <Header/>
<Route exact path="/" component={Home}/> // renders null
<Route path="/about" component={About}/> // renders <About/>
<Route path="/contact" component={Contact}/> // renders null
<Route component={Footer}/> // renders <Footer/>
```

The **exact** property will make the route match if the path property is exactly equal to the pathname

It makes sense to list a number of possible Routes next to each other,
but a **Route** component can be used anywhere to render content based on location

Grouping Routes



Routes can be grouped using a **Switch** component

```
<Switch>
  <Route exact path="/" component={Home}/>
  <Route path="/about" component={About}/>
  <Route path="/contact" component={Contact}/>
  {/* when none of the above match, <NoMatch> will be rendered */}
  <Route component={NoMatch}/>
</Switch>
```

Switch iterates over its **children** elements and
renders only the first matching Route

Path Parameters

[/:id]

It is possible to capture path parameters within a pathname

Named parameters are defined by prefixing : to the parameter name and accessed through the **match.params** prop

```
// route config
<Route path='/users/:id' component={User}/>

// component rendered by route
function User({ match }) {
  return <h3>User ID: {match.params.id}</h3>;
};
```

Query String

[/?a=b]

The query string is available through the **location.search** prop
Any query string library such as the popular **qs** can be used to parse it

```
// import the popular qs library
import qs from 'qs';

// route config
<Route path='/users' component={User}/>

// component rendered by route
function User({ location }) {

  const queryParams = qs.parse(location.search);

  // /users?username=rui
  return <h3>User Name: {queryParams.username}</h3>;
};
```

Navigation



Special components are required for navigation as standard anchor tags would cause a page refresh

The **Link** component updates the URL and changes the rendered content without reloading the page

```
// location = { pathname: '/react' }  
  
// renders <a href="/">Home</a>  
<Link to="/">Home</Link>  
  
// location = { pathname: '/react' }  
// renders <a href="/react" className="hurray">React</a>  
<NavLink to="/react" activeClassName="hurray">React</NavLink>
```

The **NavLink** component can be styled when active

Redirect



Navigation can be forced by rendering a **Redirect** component

```
function Admin({token}) {  
  if (!token) {  
    return <Redirect to='/login'/>;  
  }  
  
  return <h2>Admin Section</h2>;  
}
```

{ Exercise }

Code Awards

